

1.A Design and implement algorithms to encrypt and decrypt messages using classical substitution techniques (GUI)

```
import tkinter as tk
from tkinter import messagebox

def encrypt(text, shift):
    result = ""
    for ch in text:
        if ch.isupper():
            result += chr((ord(ch) - 65 + shift) % 26 + 65)
        elif ch.islower():
            result += chr((ord(ch) - 97 + shift) % 26 + 97)
        else:
            result += ch
    return result

def decrypt(text, shift):
    return encrypt(text, -shift)

def do_encrypt():
    try:
        out.set(encrypt(text_var.get(), int(shift_var.get())))
    except ValueError:
        messagebox.showerror("Error", "Shift must be an integer")

def do_decrypt():
    try:
        out.set(decrypt(text_var.get(), int(shift_var.get())))
    except ValueError:
        messagebox.showerror("Error", "Shift must be an integer")

root = tk.Tk()
root.title("Substitution Cipher (Caesar) - GUI")

text_var = tk.StringVar()
shift_var = tk.StringVar(value="3")
out = tk.StringVar()

tk.Label(root, text="Text").grid(row=0, column=0, padx=5, pady=5)
```

Sheth L.U.J & Sir M.V College
Subject: Cyber & Information Security

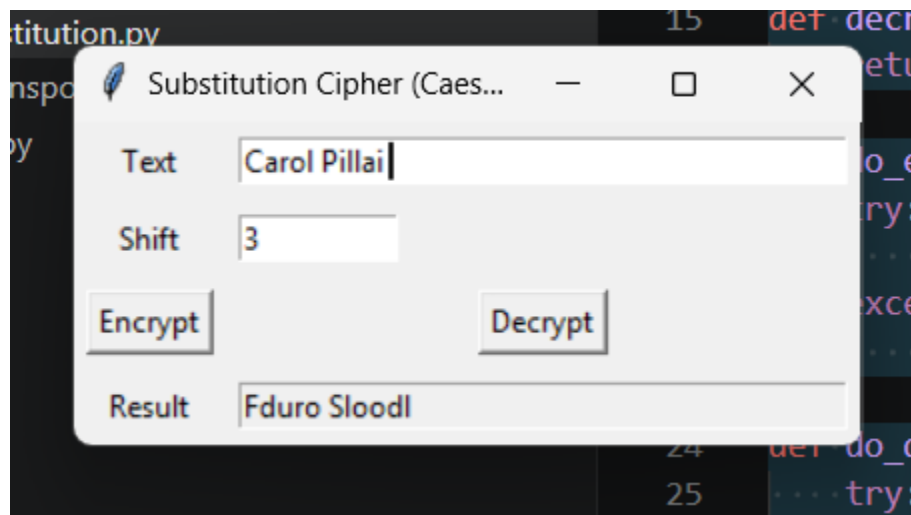
```
tk.Entry(root, textvariable=text_var, width=40).grid(row=0, column=1,
padx=5, pady=5)

tk.Label(root, text="Shift").grid(row=1, column=0, padx=5, pady=5)
tk.Entry(root, textvariable=shift_var, width=10).grid(row=1, column=1,
sticky="w", padx=5, pady=5)

tk.Button(root, text="Encrypt", command=do_encrypt).grid(row=2, column=0,
padx=5, pady=5)
tk.Button(root, text="Decrypt", command=do_decrypt).grid(row=2, column=1,
padx=5, pady=5)

tk.Label(root, text="Result").grid(row=3, column=0, padx=5, pady=5)
tk.Entry(root, textvariable=out, width=40, state="readonly").grid(row=3,
column=1, padx=5, pady=5)

root.mainloop()
```



Sheth L.U.J & Sir M.V College
Subject: Cyber & Information Security

1.A Design and implement algorithms to encrypt and decrypt messages using classical substitution techniques (CLI)

```
import sys
```

```
def encrypt(text, shift):
    result = ""
    for ch in text:
        if ch.isupper():
            result += chr((ord(ch) - 65 + shift) % 26 + 65)
        elif ch.islower():
            result += chr((ord(ch) - 97 + shift) % 26 + 97)
        else:
            result += ch
    return result

def decrypt(text, shift):
    return encrypt(text, -shift)

if __name__ == "__main__":
    if len(sys.argv) != 4:
        print("Usage: python 1A_substitution_cli.py [encrypt|decrypt] <text> <shift>")
    else:
        mode, text, shift = sys.argv[1], sys.argv[2], int(sys.argv[3])
        if mode == "encrypt":
            print(encrypt(text, shift))
        elif mode == "decrypt":
            print(decrypt(text, shift))
        else:
            print("Invalid mode")
```

```
PS C:\Users\Carol Pillai\Desktop\College\Sem 5\CYBER SECURITY> c:: cd 'c:\Users\Carol Pillai\Desktop\College\Sem 5\CYBER SECURITY'; & 'c:\Users\Carol Pillai\AppData\Local\Programs\Python\Python39\python.exe' 'c:\Users\Carol Pillai\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '54670' '--' 'c:\Users\Carol Pillai\Desktop\College\Sem 5\CYBER SECURITY\1.1cli.py'
Usage: python 1A_substitution_cli.py [encrypt|decrypt] <text> <shift>
PS C:\Users\Carol Pillai\Desktop\College\Sem 5\CYBER SECURITY> python 1.1cli.py encrypt "Hello World" 3
Khoor Zruog
PS C:\Users\Carol Pillai\Desktop\College\Sem 5\CYBER SECURITY> |
```

1. B Design and implement algorithms to encrypt and decrypt messages using transposition techniques (GUI)

```
import tkinter as tk
from tkinter import messagebox

def key_order(key):
    return sorted(range(len(key)), key=lambda i: key[i])

def encrypt(text, key):
    cols = len(key)
    rows = -(-len(text) // cols)
    padded = text.ljust(rows * cols, "X")
    grid = [padded[r * cols:(r + 1) * cols] for r in range(rows)]
    order = key_order(key)
    result = ""
    for i in order:
        for row in grid:
            result += row[i]
    return result

def decrypt(text, key):
    cols = len(key)
    rows = len(text) // cols
    order = key_order(key)
    grid = ["" for _ in range(cols)] for _ in range(rows)
    idx = 0
    for i in order:
        for r in range(rows):
            grid[r][i] = text[idx]
            idx += 1
    return "".join("".join(row) for row in grid)

def do_encrypt():
    if not key_var.get():
        messagebox.showerror("Error", "Key required")
        return
    out.set(encrypt(text_var.get(), key_var.get()))

def do_decrypt():
```

Sheth L.U.J & Sir M.V College
Subject: Cyber & Information Security

```
    if not key_var.get():
        messagebox.showerror("Error", "Key required")
        return
    out.set(decrypt(text_var.get(), key_var.get()))

root = tk.Tk()
root.title("Transposition Cipher (Columnar) - GUI")

text_var = tk.StringVar()
key_var = tk.StringVar(value="CIPHER")
out = tk.StringVar()

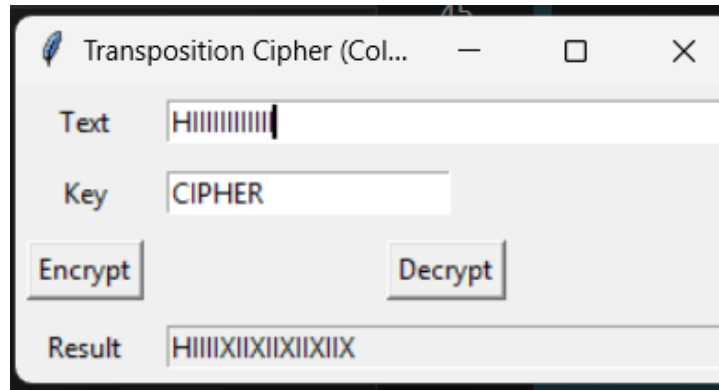
tk.Label(root, text="Text").grid(row=0, column=0, padx=5, pady=5)
tk.Entry(root, textvariable=text_var, width=40).grid(row=0, column=1,
padx=5, pady=5)

tk.Label(root, text="Key").grid(row=1, column=0, padx=5, pady=5)
tk.Entry(root, textvariable=key_var, width=20).grid(row=1, column=1,
sticky="w", padx=5, pady=5)

tk.Button(root, text="Encrypt", command=do_encrypt).grid(row=2, column=0,
padx=5, pady=5)
tk.Button(root, text="Decrypt", command=do_decrypt).grid(row=2, column=1,
padx=5, pady=5)

tk.Label(root, text="Result").grid(row=3, column=0, padx=5, pady=5)
tk.Entry(root, textvariable=out, width=40, state="readonly").grid(row=3,
column=1, padx=5, pady=5)

root.mainloop()
```



1. B Design and implement algorithms to encrypt and decrypt messages using transposition techniques (CLI)

```
import sys

def key_order(key):
    return sorted(range(len(key)), key=lambda i: key[i])

def encrypt(text, key):
    cols = len(key)
    rows = -(-len(text) // cols)
    padded = text.ljust(rows * cols, "X")
    grid = [padded[r * cols:(r + 1) * cols] for r in range(rows)]
    order = key_order(key)
    result = ""
    for i in order:
        for row in grid:
            result += row[i]
    return result

def decrypt(text, key):
    cols = len(key)
    rows = len(text) // cols
    order = key_order(key)
    grid = ["" for _ in range(cols)] for _ in range(rows)
    idx = 0
```

Sheth L.U.J & Sir M.V College
Subject: Cyber & Information Security

```
for i in order:
    for r in range(rows):
        grid[r][i] = text[idx]
        idx += 1
    return "".join("".join(row) for row in grid)

if __name__ == "__main__":
    if len(sys.argv) != 4:
        print("Usage: python 1B_transposition_cli.py [encrypt|decrypt]
<text> <key>")
    else:
        mode, text, key = sys.argv[1], sys.argv[2], sys.argv[3]
        if mode == "encrypt":
            print(encrypt(text, key))
        elif mode == "decrypt":
            print(decrypt(text, key))
        else:
            print("Invalid mode")
```

```
53999' '--' 'c:\Users\Carol Pillai\Desktop\College\Sem 5\CYBER SECURITY\2bcli.py'
Usage: python 1B_transposition_cli.py [encrypt|decrypt] <text> <key>
● PS C:\Users\Carol Pillai\Desktop\College\Sem 5\CYBER SECURITY> python 2bcli.py encrypt "Hello World" 3
Hello World
○ PS C:\Users\Carol Pillai\Desktop\College\Sem 5\CYBER SECURITY> |
```